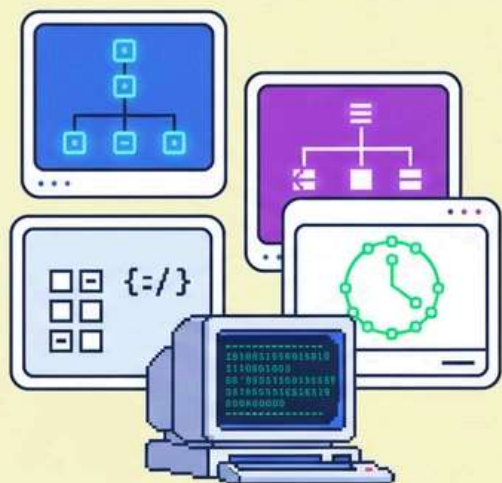




10 Secret Tips That Make You Better at DSA



Akash Pattnaik

Posted on 12 Jan



1



1



1



2



1

10 Secret Tips That Make You Better at DSA

#algorithms #beginners #programming #productivity

Introduction

Data Structures and Algorithms (DSA) often feel overwhelming at first. With so many concepts, patterns, and problem types, it's easy to feel stuck or slow. But here's the good news: becoming good at DSA is less about talent and more about strategy. The right habits, mindset, and techniques can dramatically speed up your progress. In this article, you'll discover **10 secret tips that can make you significantly better at DSA**, even if you're starting from scratch.

Table of Contents

1. Master the Basics Before Chasing Advanced Topics
2. Think in Patterns, Not Problems

3. Always Dry Run Your Code
 4. Learn Time and Space Complexity Visually
 5. Write Code on Paper (Yes, Really!)
 6. Solve Fewer Problems, but Solve Them Deeply
 7. Revisit Old Problems Regularly
 8. Explain Your Solution Out Loud
 9. Learn When *Not* to Use a Data Structure
 10. Build Consistency, Not Motivation
-

1. Master the Basics Before Chasing Advanced Topics

Many learners rush into advanced topics like dynamic programming or graphs without fully understanding arrays, strings, and linked lists. This creates weak foundations. Spend time mastering basics like loops, recursion, and simple data structures. Strong fundamentals make complex topics feel easier and more logical.

2. Think in Patterns, Not Problems

Most DSA problems are variations of a few core patterns such as sliding window, two pointers, recursion, backtracking, or divide and conquer. Instead of memorizing solutions, train your brain to recognize these patterns. Once you spot the pattern, the solution becomes much clearer.

3. Always Dry Run Your Code

Before jumping into coding, dry run the logic step by step with a small example. This helps you catch edge cases, logical errors, and unnecessary steps early. Dry running also improves your problem-solving clarity and reduces debugging time.

4. Learn Time and Space Complexity Visually

Big-O notation can feel abstract. Instead of memorizing formulas, visualize how your algorithm grows with input size. Ask questions like: *How many loops are running?* or *Is extra memory being used?* This approach makes complexity analysis intuitive rather than confusing.

5. Write Code on Paper (Yes, Really!)

Writing code on paper forces you to slow down and think clearly. It improves logical flow and prepares you for interviews where you may not have an editor or auto-complete. This habit strengthens your confidence and precision.

6. Solve Fewer Problems, but Solve Them Deeply

Solving 100 problems poorly is less effective than solving 30 problems deeply. For every problem you solve:

- Understand why the solution works
- Explore alternative approaches
- Analyze time and space complexity

Depth beats quantity in DSA learning.

7. Revisit Old Problems Regularly

Re-solving old problems after a few weeks is incredibly powerful. You'll notice improved speed, better logic, and cleaner code. This reinforces long-term memory and helps patterns stick naturally.

8. Explain Your Solution Out Loud

If you can explain your solution clearly, you truly understand it. Try teaching the solution to a friend, or even to yourself. This technique exposes gaps in understanding and improves interview communication skills.

9. Learn When *Not* to Use a Data Structure

Many beginners overuse complex data structures when simpler ones work better. Always ask: *Is this the simplest solution?* Choosing the right (or minimal) data structure often leads to cleaner and faster code.

10. Build Consistency, Not Motivation

Motivation comes and goes, but consistency builds skill. Even 30 minutes of daily DSA practice is better than long, irregular sessions. Create a routine and stick to it. Progress will follow naturally.

Final Thoughts

Becoming better at DSA isn't about shortcuts—it's about smart learning. Apply these 10 secret tips consistently, and you'll notice improved problem-solving speed, better confidence, and stronger fundamentals. Remember, every expert was once a beginner who didn't give up.



Postmark

PROMOTED



Find out why “Postmark = ”

(Their words, not ours)



“Postmark blows every other transactional email product out of the water.”

Kyle Racki, CEO

Josh Sommers, Lead Developer

[Get started free](#)



[See why Postmark = !\[\]\(6bb0e4f14c4133b37d2887cb37e67ddd_img.jpg\)](#)

[Get started free](#)

[Read More](#)

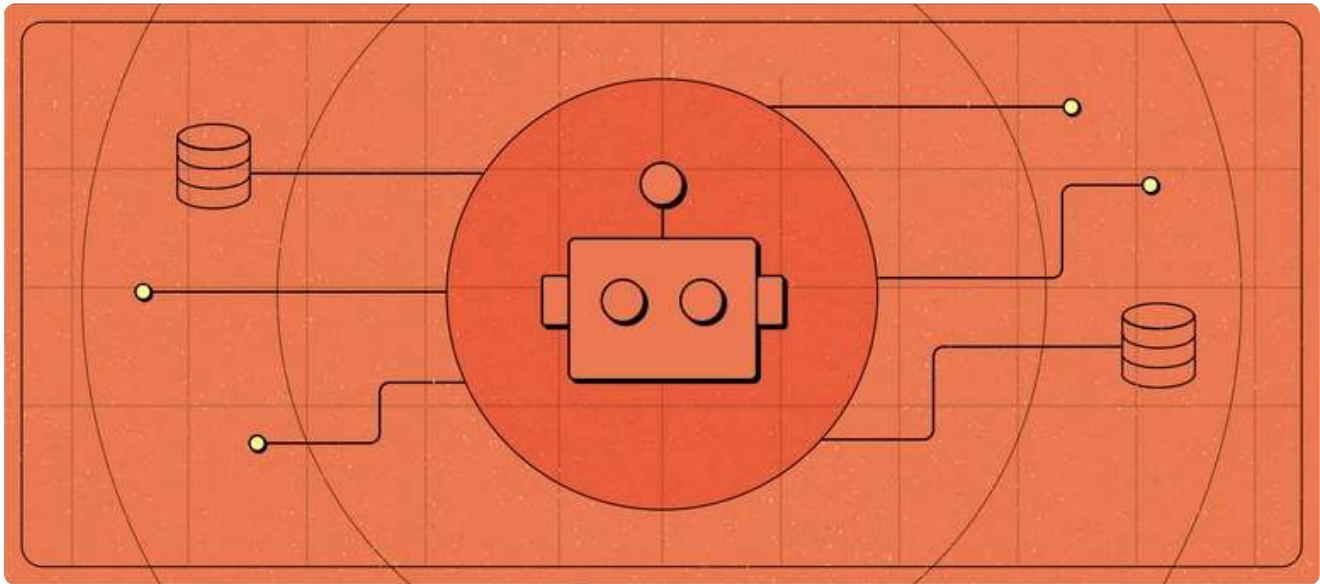
Top comments (0)

[Code of Conduct](#) • [Report abuse](#)



Tiger Data (Creators of TimescaleDB)

PROMOTED



[An MCP Server That Actually Understands Postgres](#)

We took 10+ years of Postgres experience and turned it into built-in prompts. Agents get tools for schema design, query optimization, and migrations, plus they can search Postgres docs on the fly.

[Learn more](#)**Akash Pattnaik**

20 | Developer turned vibe coder | Student

LOCATION

India

EDUCATION

Siksha 'O' Anusandhan ITER

PRONOUNS

he/him

JOINED

27 Dec 2021

More from Akash Pattnaik

Why Flutter beats React Native ?

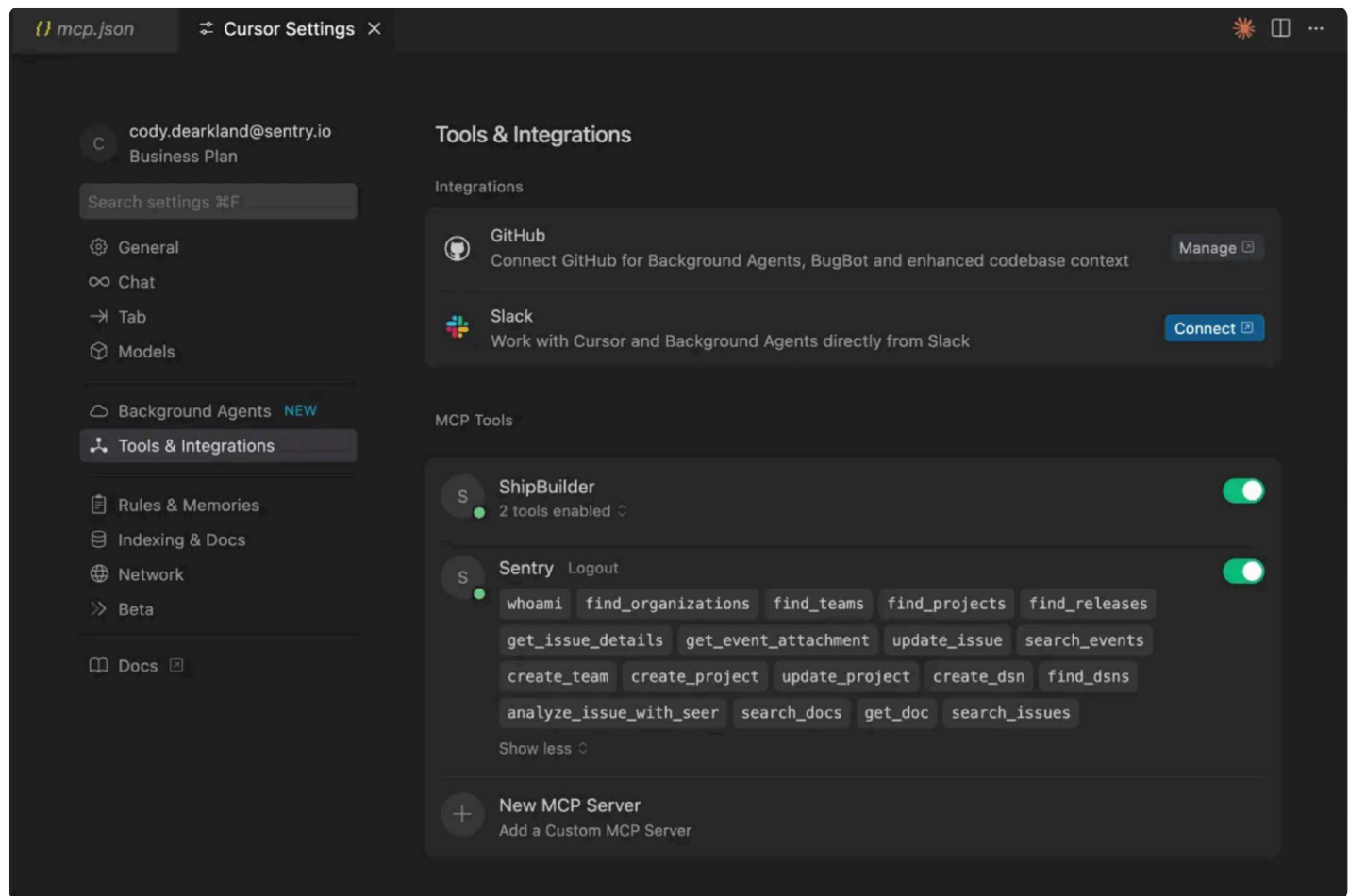
#flutter #beginners #programming #reactnative

End of ElephantSQL 🐘

#news #programming #database #discuss

The new BottomBar in Flutter

#flutter #tutorial #beginners #productivity



[Tutorial: Debugging with Cursor + Sentry MCP](#)

No more copying and pasting error messages, logs, or trying to describe your distributed tracing setup or stack traces in chat. MCP can investigate real issues, understand their impact, and suggest fixes based on the actual production context.

 [See how →](#)